

1 Exercice : Analyse de performances

Dans cette exercice, nous allons lancer une évaluation du matériel et de l'implémentation MPI sur le calculateur à disposition concernant les performances de recouvrements des communications collectives non-bloquantes. Nous analyserons ensuite les résultats. Pour se faire, nous allons utiliser les *benchmarks* implémentés par Intel :

- (a) Chargez une version de MPI sur le calculateur à l'aide de `module load`.
- (b) Depuis le calculateur, récupérez le projet git des **benchmarks** à l'adresse suivante : <https://github.com/intel/mpi-benchmarks.git>.
- (c) Configurez les variables d'environnements des compilateurs comme suit :

```
export CXX=mpi++  
export CC=mpicc
```
- (d) Lancez le *makefile* avec pour option IMB-NBC.
- (e) Lancez cette évaluation à l'aide de la commande `srun` comme suit :

```
srun -N <NB_NOEUDS> -n <NB_COEURS> -p <NOM_PARTITION> IMB-NBC -nppmin <NB_NOEUDS_MIN>
```

L'option `-nppmin` permet de spécifier le nombre minimal de processus MPI utilisé pour évaluer chaque communication.
Attention, cette évaluation peut prendre un peu de temps !
- (f) Analysez comment est calculé le recouvrement (*overlap*) des communications¹.
- (g) Que pouvez-vous dire sur les performances des communications collectives non-bloquantes sur le calculateur à disposition ?

2 Exercice : Pipeline de réductions non-bloquantes

Dans le fichier `Pipeline/pipeline.c`, nous allons faire une réduction de trois grandeurs définies sur trois vecteurs différents, potentiellement très grands. Pour cela, nous allons comparer 4 approches différentes. La taille des vecteurs, locale à chaque processus, est à fixer par l'utilisateur à l'exécution comme suit :

```
srun -n <NB_COEURS> <NOM_EXECUTABLE> <TAILLE_VECTEUR>
```

- (a) Première option : faire la réduction en mode bloquant, directement sur le vecteur des trois grandeurs. Complétez pour cela la **Zone-2-1-A**. Relevez le temps d'exécution pour différentes tailles de maillage et de nombre de processus.
- (b) Deuxième option : faire la réduction en mode bloquant, sur chacune des trois grandeurs. Complétez pour cela la **Zone-2-1-B**. Relevez le temps d'exécution pour différentes tailles de maillage et de nombre de processus.
- (c) Troisième option : faire la réduction en mode non-bloquant, directement sur le vecteur des trois grandeurs. Complétez pour cela la **Zone-2-1-C**. Relevez le temps d'exécution pour différentes tailles de maillage et de nombre de processus.
- (d) Quatrième option : faire la réduction en mode non-bloquant, sur chacune des trois grandeurs. Complétez pour cela la **Zone-2-1-D**. L'intérêt est alors de lancer les trois opérations, puis de les finaliser ensemble. Relevez le temps d'exécution pour différentes tailles de maillage et de nombre de processus.

¹On pourra s'aider du manuel d'utilisateur disponible à l'adresse suivante : <https://www.intel.com/content/www/us/en/docs/mpi-library/user-guide-benchmarks/2021-2/overview.html>

- (e) Commentez les résultats obtenus en terme de performances.
- (f) Les valeurs numériques de la réduction devraient être toujours les mêmes, quelque soit la composante, mais aussi la taille du maillage et le nombre de processus.
Qu'observez-vous ? Pouvez-vous l'expliquer ?

3 Exercice : Compréhension

Dans cet exercice, nous allons voir différents exemples de ce que permet la norme, ce qu'elle interdit, ou ce sur quoi elle ne statue pas.

Question1

Mélange de collectives et de communications points-à-points. Commentez le code suivant :

```

1      MPI_Request reqs[2];
2      switch(rank)
3      {
4          case 0:
5              MPI_Ibarrier(comm, &reqs[0]);
6              MPI_Send(buf, count, dtype, 1, tag, comm);
7              MPI_Wait(&reqs[0], MPI_STATUS_IGNORE);
8              break;
9          case 1:
10             MPI_Irecv(buf, count, dtype, 0, tag, comm, &reqs[0]);
11             MPI_Ibarrier(comm, &reqs[1]);
12             MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);
13             break;
14     }
```

Question2

Détail d'implémentation. Commentez le code suivant :

```

1      switch(rank) {
2          case 0:
3              MPI_Bcast(buf1, count, type, 0, comm);
4              MPI_Bcast(buf2, count, type, 1, comm);
5              break;
6          case 1:
7              MPI_Bcast(buf2, count, type, 1, comm);
8              MPI_Bcast(buf1, count, type, 0, comm);
9              break;
10     }
```

4 Exercice : Topologie et communications collectives

Ce premier exercice est consacré à l'usage des collectives non-bloquantes, dans le contexte d'un communicateur global, puis d'une topologie.

Question1

Cette première question est une mise en pratique des commandes MPI pour la création d'une topologie cartésienne. Dans le fichier `Grid/grid.c`, nous souhaitons construire un maillage décomposé en sous-domaines suivant une topologie cartésienne.

- (a) Complétez la portion de code `Zone-4-1-A` afin de créer, en utilisant `MPI_Dims_create` et `MPI_Cart_create`², une topologie cartésienne adaptée au nombre de processus MPI.

²Ne jamais hésiter à lire les pages de manuel d'un sous-programme MPI en ligne avec `man MPI-<nom_programme>` sur un terminal !

- (b) Complétez la portion de code **Zone-4-1-B** dans laquelle chaque processus doit calculer les dimensions de son sous-domaine, vous aurez besoin de `MPI_Cart_coords`. Calculez aussi le nombre de mailles du sous-domaine, et vérifiez que la somme des nombres de mailles de tous les sous-domaines donne bien le nombre total de mailles (utilisez `MPI_Reduce`).

Question2

- (a) Maintenant, nous souhaitons que chaque processus ait un tableau du nombre de mailles dans l'ensemble des autres processus. Complétez la portion de code **Zone-4-2-A** avec un `all_gather` du nombre de maille sur l'ensemble des processus.
- (b) Dans la **Zone-4-2-B**, réalisez un `all_gather` sur la topologie (utilisez `MPI_Neighbor_allgather`).
- (c) Existe-t-il un moyen de réaliser une communication `all_gather` dans sa version non-bloquante en la recouvrant par du calcul sur mailles ?
- (d) Si oui, implémentez les versions non-bloquantes du `all_gather` sur le communicateur global et celui sur la topologie (utilisez `MPI_Ineighbor_allgather`)
- (e) Comparez les performances sur les deux communicateurs.